2

FINDING PALINGRAM SPELLS

Radar. Kayak. Rotator. Sexes. What do these words all have in common? They're *palindromes*, words that are spelled the same forward and backward. Even better are *palingrams*, whole phrases that behave the same way. Napoleon is the author of the most famous palingram. When he first saw Elba, the island of his exile, he said, "Able was I ere I saw Elba."

In 2011, DC Comics published an interesting story that made clever use of palingrams. The superhero sorceress Zatanna was cursed so that she could cast spells only by speaking palindromically. She managed to think up just enough two-word phrases like *nurses run*, *stack cats*, and *puff up* to defeat her sword-wielding attacker. This got me wondering: just how many "combative" palingrams are there? And are there better choices for Zatanna?

In this chapter, you'll load dictionary files from the internet and use Python to discover first palindromes and then the more complex palingrams in those files. Then you'll use a tool called `cProfile` to analyze your palingram code so that you can make it more performant. Finally, you'll sift through the palingrams to see how many have an "aggressive" nature.

Finding and Opening a Dictionary

All the projects in this chapter require a listing of words in a text file format, commonly referred to as a *dictionary file*, so let's start by learning how to load one.

Despite their name, dictionary files contain only words—no pronunciation, syllable count, definitions, and so on. This is good news, as those things would just get in our way. And even better, dictionary files are available online for free.

You can find suitable dictionary files at the locations listed in **Table 2-1**. Download one of the files or, if it opens directly, copy and paste the contents into a text editor like Notepad or WordPad (TextEdit on macOS) and save it as a *.txt* file. Keep the dictionary in the same folder as the Python code. I used the *2of4brif.txt* file to prepare this project. It can be found in the downloadable *12dicts-6.0.2.zip* file on the website listed first in **Table 2-1**.

Table 2-1: Downloadable Dictionary Files

File	Number of words
http://wordlist.aspell.net/12dicts/	60,388
https://inventwithpython.com/dictionary.txt	45,000
http://www-personal.umich.edu/~jlawler/wordlist.html	69,903
http://greenteapress.com/thinkpython2/code/words.txt	113,809

In addition to the files in **Table 2-1**, Unix and Unix-like operating systems come packaged with a large newline-delimited word file of more than 200,000 words. It is usually stored in */usr/share/dict/words* or */usr/dict/words*. On Debian GNU/Linux, word lists are in */usr/share/opendict/dictionaries*. The macOS dictionaries are generally found in */Library/Dictionaries*, and non-English dictionaries are included.

You may need to do an online search for your operating system and version to find the exact directory path if you want to use one of these files.

Some dictionary files exclude *a* and *I* as words. Others may include every letter in the dictionary as a single word "header" (such as *d* at the start of words beginning with *d*). We'll ignore one-letter palindromes in these projects, so these issues shouldn't be a problem.

Handling Exceptions When Opening Files

Whenever you load an external file, your program should automatically check for I/O issues, like missing files or incorrect filenames, and let you know if there is a problem.

Use the following `try` and `except` statements to catch and handle *exceptions*, which are errors detected during execution:

```
❶ try:
    ❷ with open(file) as in_file:
            do something
   except IOError❸ as e:
    ❹ print("{}\nError opening {}. Terminating program.".format(e, file),
              file=sys.stderr)
    ❺ sys.exit(1)
```

The `try` clause is executed first ❶. The `with` statement will automatically close the file after the nested block of code, regardless of how the block exits ❷. Closing files prior to terminating a process is a good practice. If you don't close those files, you could run out of file descriptors (mainly a problem with large scripts that run for a long time), lock the file from further access in Windows, corrupt the files, or lose data if you are writing to the file.

If something goes wrong and if the type of error matches the exception named after the `except` keyword ❸, the rest of the `try` clause is skipped, and the `except` clause is executed ❹. If nothing goes wrong, the `try` clause is executed, and the `except` clause is skipped. The `print` statement in the `except` clause lets you know there's a problem, and the `file=sys.stderr` argument colors the error statement red in the IDLE interpreter window.

The `sys.exit(1)` ❺ statement is used to terminate the program. The `1` in `sys.exit(1)` indicates that the program experienced an error and did not close successfully.

If an exception occurs that *doesn't* match the named exception in the `except` clause, it is passed to any outer `try` statements or the main program execution. If no handler is found, the *unhandled exception* causes the program to stop with a standard "traceback" error message.

Loading the Dictionary File

Listing 2-1 loads a dictionary file as a list. Manually enter this script or download it as *load_dictionary.py* from **https://nostarch.com/impracticalpython/**.

You can import this file into other programs as a module and run it with a one-line statement. Remember, a module is simply a Python program that can be used in another Python program. As you're probably aware, modules represent a form of *abstraction*. Abstraction means you don't have to worry about all the coding details. A principle of abstraction is *encapsulation*, the act of hiding the details. We encapsulate the file-loading code in a module so you don't have to see or worry about the detailed code in another program.

load_dictionary.py

```
    """Load a text file as a list.

    Arguments:
    -text file name (and directory path, if needed)

    Exceptions:
    -IOError if filename not found.

    Returns:
    -A list of all words in a text file in lower case.

    Requires-import sys
```

```
    """
❶ import sys

❷ def load(file):
        """Open a text file & return a list of lowercase strings."""
        try:
            with open(file) as in_file:
             ❸ loaded_txt = in_file.read().strip().split('\n')
             ❹ loaded_txt = [x.lower() for x in loaded_txt]
                return loaded_txt
        except IOError as e:
         ❺ print("{}\nError opening {}. Terminating program.".format(e, file),
                file=sys.stderr)
            sys.exit(1)
```

Listing 2-1: The module for loading a dictionary file as a list

After the docstring, we import system functions with `sys` so that our error-handling code will work ❶. The next block of code defines a function based on the previous file-opening discussion ❷. The function takes a filename as an argument.

If no exceptions are raised, the text file's whitespace is removed, and its items are split into separate lines and added to a list ❸. We want each word to be a separate item in the list, before the list is returned. And since case matters to Python, the words in the list are converted to lowercase via *list comprehension* ❹. List comprehension is a shorthand way to convert a list, or other iterable, into another list. In this case, it replaces a `for` loop.

If an I/O error is encountered, the program displays the standard error message, designated by the `e`, along with a message describing the event and informing the user that the program is ending ❺. The `sys.exit(1)` command then terminates the program.

This code example is for illustrative purposes, to show how these steps work together. Generally, you wouldn't call `sys.exit()` from a module, as you may want your program to do something—like write a log file—prior

to terminating. In later chapters, we'll move both the `try-except` blocks and `sys.exit()` into a `main()` function for clarity and control.

Project #2: Finding Palindromes

You'll start by finding single-word palindromes in a dictionary and then move on to the more difficult palindromic phrases.

THE OBJECTIVE

Use Python to search an English language dictionary file for palindromes.

The Strategy and Pseudocode

Before you get into the code, step back and think about what you want to do conceptually. Identifying palindromes is easy: simply compare a word to itself sliced backward. Here is an example of slicing a word front to back and then back to front:

```
>>> word = 'NURSES'
>>> word[:]
'NURSES'
>>> word[::-1]
'SESRUN'
```

If you don't provide values when slicing a string (or any sliceable type), the default is to use the start of the string, the end of the string, and a positive step equal to `1`.

Figure 2-1 illustrates the reverse slicing process. I've provided a starting position of `2` and a step of `-1`. Because no end index is provided (there is no index or space between the colons), the implication is to go backward (because the index step is `-1`) until there are no more characters left.

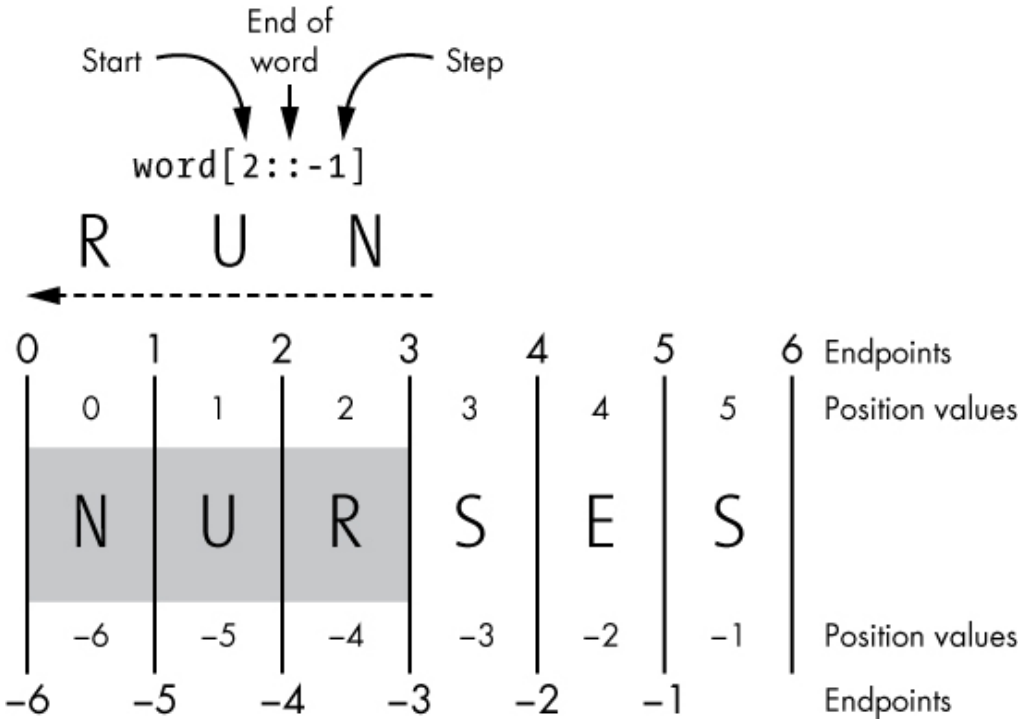

Figure 2-1: An example of negative slicing for `word = 'NURSES'`

Negative slicing doesn't behave exactly the same way as forward slicing, and the positive and negative position values and endpoints are asymmetrical. This can lead to confusion, so let's restrict our negative slicing to the simple `[::-1]` format.

Finding palindromes in the dictionary will take fewer lines of code than loading the dictionary file! Here's the pseudocode:

```
Load digital dictionary file as a list of words
Create an empty list to hold palindromes
Loop through each word in the word list:
    If word sliced forward is the same as word sliced backward:
        Append word to palindrome list
Print palindrome list
```

The Palindrome Code

Listing 2-2, *palindromes.py*, reads in an English dictionary file, identifies which words are palindromes, saves them to a list, and prints the list as stacked items. You can download this code from the book's resources at **https://www.nostarch.com/impracticalpython/**. You will also need *load_dictionary.py* and a dictionary file; save all three files in the same folder.

palindromes.py

```
    """Find palindromes (letter palingrams) in a dictionary file."""
❶ import load_dictionary
❷ word_list = load_dictionary.load('2of4brif.txt')
❸ pali_list = []

❹ for word in word_list:
        if len(word) > 1 and word == word[::-1]:
            pali_list.append(word)

    print("\nNumber of palindromes found = {}\n".format(len(pali_list)))
❺ print(*pali_list, sep='\n')
```

Listing 2-2: Finds palindromes in loaded dictionary file

Start by importing *load_dictionary.py* as a module ❶. Note that the *.py* extension is not used for importing. Also, the module is in the same folder as this script, so we don't have to specify a directory path to the module. And since the module contains the required `import sys` line, we don't need to repeat it here.

To populate our word list with words from the dictionary, call the `load()` function in the `load_dictionary` module with dot notation ❷. Pass it the name of the external dictionary file. Again, you don't need to specify a path if the dictionary file is in the same folder as the Python script. The filename you use may be different depending on the dictionary you downloaded.

Next, create an empty list to hold the palindromes ❸ and start looping through every word in `word_list` ❹, comparing the forward slice to the reverse slice. If the two slices are identical, append the word to `pali_list`. Notice that only words with more than one letter are allowed (`len(word) > 1`), which follows the strictest definition of a palindrome.

Finally, print the palindromes in an attractive way—stacked and with no quotation marks or commas ❺. You can accomplish this by looping through every word in the list, but there is a more efficient way to do it. You can use the *splat* operator (designated by the `*`), which takes a list as input and expands it into positional arguments in the function call. The

last argument is the separator used between multiple list values for printing. The default separator is a space (`sep=' '`), but instead, print each item on a new line (`sep='\n'`).

Single-word palindromes are rare, at least in English. Using a 60,000-word dictionary file, you'll be lucky to find about 60, or only 0.1 percent of all the words. Despite their rarity, however, they're easy enough to find with Python. So, let's move on to the more interesting, and more complicated, palingrams.

Project #3: Finding Palingrams

Finding palingrams requires a bit more effort than finding one-word palindromes. In this section, we'll plan and write code to find word-pair palingrams.

THE OBJECTIVE

Use Python to search an English language dictionary for two-word palingrams. Analyze and optimize the palingram code using the cProfile tool.

The Strategy and Pseudocode

Example word-pair palingrams are *nurses run* and *stir grits*. (In case you're wondering, grits are a ground-corn breakfast dish, similar to Italian polenta.)

Like palindromes, palingrams read the same forward and backward. I like to think of these as a *core* word, like *nurses*, from which a *palindromic sequence* and *reversed word* are derived (see **Figure 2-2**).

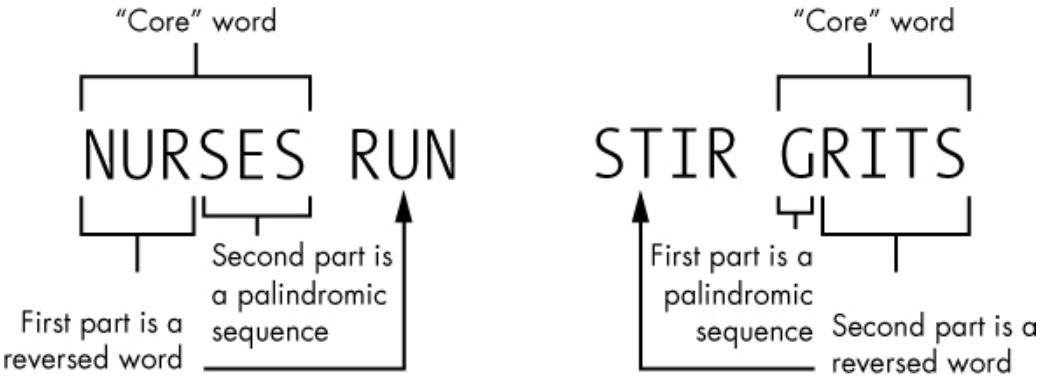

Figure 2-2: Dissecting word-pair palingrams

Our program will examine the core word. Based on **Figure 2-2**, we can make the following inferences about the core word:

1. It can have either an odd or even number of letters.
2. One contiguous part of the word spells a real word when read backward.
3. This contiguous part can occupy part or all of the core word.
4. The other contiguous part contains a palindromic sequence of letters.
5. The palindromic sequence can occupy part or all of the core word.
6. The palindromic sequence does not have to be a real word (unless it occupies the *whole* word).
7. The two parts cannot overlap or share letters.
8. The sequence is reversible.

NOTE

If the reversed word occupies the whole core word and is not a palindrome, it's called a semordnilap. *A semordnilap is similar to a palindrome except for one key difference: rather than spelling the* same *word when read backward, it spells a* different *word. Examples are* bats *and* stab, *and* wolf *and* flow. *Semordnilap, by the way, is* palindromes *spelled backward.*

Figure 2-3 represents an arbitrary word of six letters. The Xs represent the part of the word that *might* form a real word when read backward (like *run* in *nurses*). The Os represent the *possible* palindromic sequence (like *ses* in *nurses*). The word represented in the left column in **Figure 2-3** behaves like *nurses* in **Figure 2-2**, with the reversed word at the start. The word represented by the right column behaves like *grits*, with the reversed word at the end. Note that the number of combinations in each

column is the total number of letters in the word plus one; note too that the top and bottom rows represent an identical circumstance.

The top row in each column represents a semordnilap. The bottom row in each represents a palindrome. These are both reversed words, just different *types* of reversed words. Hence, they count as one entity and both can be identified with a single line of code in a single loop.

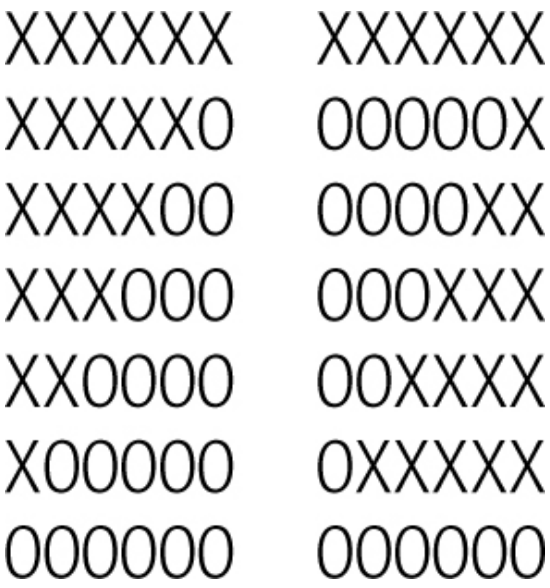

Figure 2-3: Possible positions for letters of the reversed word (X) and the palindromic sequence (O) in a six-letter core word

To see the diagram in action, consider **Figure 2-4**, which shows the palingrams *devils lived* and *retro porter*. *Devils* and *porter* are both core words and mirror images of each other with respect to palindromic sequences and reversed words. Compare this to the semordnilap *evil* and the palindrome *kayak*.

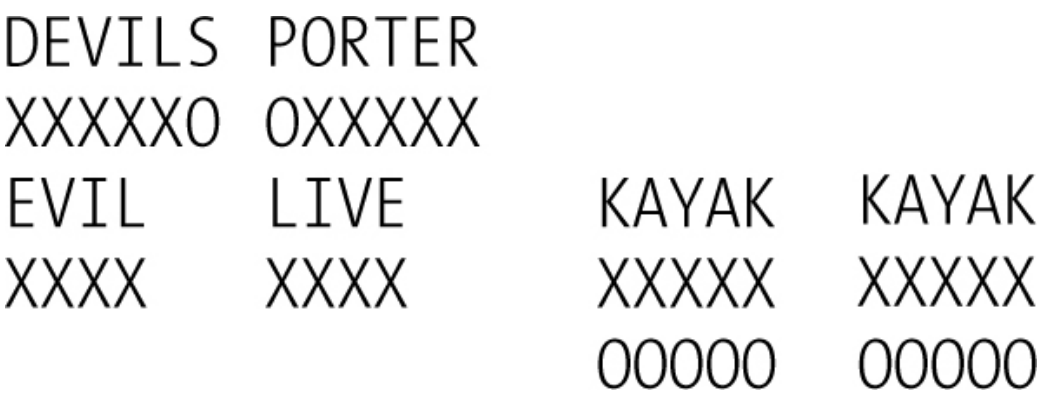

Figure 2-4: Reversed words (Xs) and palindromic sequences (Os) in words, semordnilaps, and palindromes.

Palindromes are both reversed words *and* palindromic sequences. Since they have the same pattern of Xs as in semordnilaps, they can be handled with the same code used for semordnilaps.

From a strategy perspective, you'll need to loop through each word in the dictionary and evaluate it for *all of the combinations* in **Figure 2-3**. Assuming a 60,000-word dictionary, the program will need to take about 500,000 passes.

To understand the loops, take a look at the core word for the palingram *stack cats* in **Figure 2-5**. Your program needs to loop through the letters in the word, starting with an end letter and adding a letter with each iteration. To find palingrams like *stack cats*, it will simultaneously evaluate the word for the presence of a palindromic sequence at the end of the core word, *stack*, and a reversed word at the start. Note that the first loop in **Figure 2-5** will be successful, as a single letter (*k*) can serve as a palindrome in this situation.

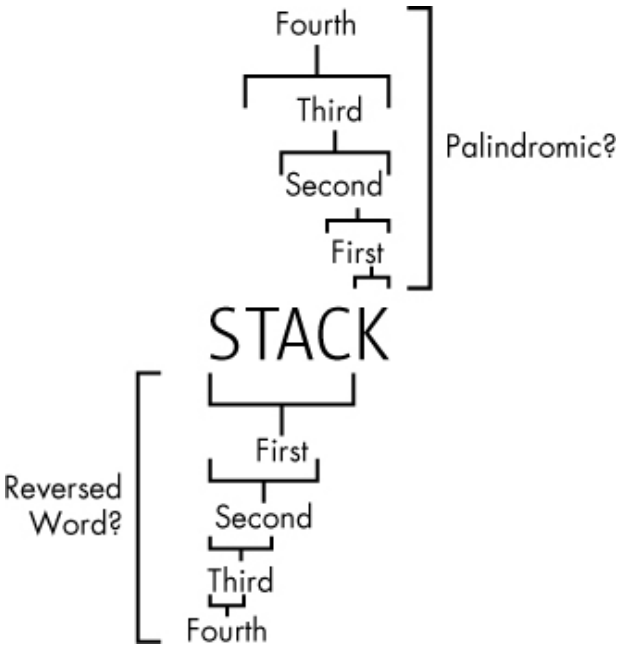

Figure 2-5: Example loops through a core word, simultaneously looking for palindromes and reversed words

But you're not through yet. To capture the "mirror image" behavior in **Figure 2-3**, you have to run the loops in reverse, looking for palindromic sequences at the start of the word and reversed words at the end. This will allow you to find palingrams like *stir grits*.

Here is the pseudocode for a palingram-finding algorithm:

```
Load digital dictionary as a list of words
Start an empty list to hold palingrams
For word in word list:
    Get length of word
    If length > 1:
        Loop through the letters in the word:
            If reversed word fragment at front of word is in word list and letter
            after form a palindromic sequence:
               Append word and reversed word to palingram list
            If reversed word fragment at end of word is in word list and letters
            before form a palindromic sequence:
               Append reversed word and word to palingram list
Sort palingram list alphabetically
Print word-pair palingrams from palingram list
```

The Palingrams Code

Listing 2-3, *palingrams.py*, loops through a word list, identifies which words form word-pair palingrams, saves those pairs to a list, and prints the list as stacked items. You can download the code from **https://www.nostarch.com/impracticalpython/**. I suggest you use the *2of4brif.txt* dictionary file to start so that your results will match mine. Store your dictionary and *load_dictionary.py* in the same folder as the palingrams script.

palingrams.py

```
    """Find all word-pair palingrams in a dictionary file."""
    import load_dictionary

    word_list = load_dictionary.load('2of4brif.txt')

    # find word-pair palingrams
❶ def find_palingrams():
        """Find dictionary palingrams."""
        pali_list = []
        for word in word_list:
         ❷ end = len(word)
```

```
        ❸ rev_word = word[::-1]
        ❹ if end > 1:
            ❺ for i in range(end):
                ❻ if word[i:] == rev_word[:end-i] and rev_word[end-i:] in word_li
                        pali_list.append((word, rev_word[end-i:]))
                ❼ if word[:i] == rev_word[end-i:] and rev_word[:end-i] in word_li
                        pali_list.append((rev_word[:end-i], word))
    ❽ return pali_list

❾ palingrams = find_palingrams()
  # sort palingrams on first word
  palingrams_sorted = sorted(palingrams)

  # display list of palingrams
❿ print("\nNumber of palingrams = {}\n".format(len(palingrams_sorted)))
  for first, second in palingrams_sorted:
      print("{} {}".format(first, second))
```

Listing 2-3: Finds and prints word-pair palingrams in loaded dictionary

After repeating the steps you used in the *palindromes.py* code to load a dictionary file, define a function to find palingrams ❶. Using a function will allow you to isolate the code later and time how long it takes to process all the words in the dictionary.

Immediately start a list called `pali_list` to hold all the palingrams the program discovers. Next, start a `for` loop to evaluate the words in `word_list`. For each word, find its length and assign its length to the variable `end` ❷. The word's length determines the indexes the program uses to slice through the word, looking for every possible reversed word-palindromic sequence combination, as in **Figure 2-3**.

Next, negatively slice through the word and assign the results to the variable `rev_word` ❸. An alternative to `word[::-1]` is `''.join(reversed(word))`, which some consider more readable.

Since you are looking for word-pair palingrams, exclude single-letter words ❹. Then nest another `for` statement to loop through the letters in the current word ❺.

Now, run a conditional requiring the back end of the word to be palindromic and the front end to be a reverse word in the word list (in other words, a "real" word) ❻. If a word passes the test, it is appended to the palingram list, immediately followed by the reversed word.

Based on **Figure 2-3**, you know you have to repeat the conditional, but change the slicing direction and word order to reverse the output. In other words, you must capture palindromic sequences at the start of the word rather than at the end ❼. Return the list of palingrams to complete the function ❽.

With the function defined, call it ❾. Since the order in which dictionary words are added to the palingram list switches during the loop, the palingrams won't be in alphabetical order. So, sort the list so that the first words in the word pair are in alphabetical order. Print the length of the list ❿, then print each word-pair on a separate line.

As written, *palingrams.py* will take about three minutes to run on a dictionary file with about 60,000 words. In the next sections, we'll investigate the cause of this long runtime and see what we can do to fix it.

Palingram Profiling

Profiling is an analytical process that gathers statistics on a program's behavior—for example, the number and duration of function calls—as the program executes. Profiling is a key part of the optimization process. It tells you exactly what parts of a program are taking the most time or memory. That way, you'll know where to focus your efforts to improve performance.

Profiling with cProfile

A *profile* is a measurement output—a record of how long and how often parts of a program are executed. The Python standard library provides a handy profiling interface, `cProfile`, which is a C extension suitable for profiling long-running programs.

Something in the `find_palingrams()` function probably accounts for the relatively long runtime of the *palingrams.py* program. To confirm, let's run `cProfile`.

Copy the following code into a new file named *cprofile_test.py* and save it in the same folder as *palingrams.py* and the dictionary file. This code imports `cProfile` and the palingrams program, and it runs `cProfile` on the `find_palingrams()` function—called with dot notation. Note again that you don't need to specify the *.py* extension.

```
import cProfile
import palingrams
cProfile.run('palingrams.find_palingrams()')
```

Run *cprofile_test.py* and, after it finishes (you will see the `>>>` in the interpreter window), you should see something similar to the following:

```
         62622 function calls in 199.452 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000  199.451  199.451 <string>:1(<module>)
        1  199.433  199.433  199.451  199.451 palingrams.py:7(find_palingrams)
        1    0.000    0.000  199.452  199.452 {built-in method builtins.exec}
    60388    0.018    0.000    0.018    0.000 {built-in method builtins.len}
     2230    0.001    0.000    0.001    0.000 {method 'append' of 'list' objects}
```

All that looping, slicing, and searching took 199.452 seconds on my machine, but of course your times may differ from mine. You also get additional information on some of the built-in functions, and since each palingram called the built-in `append()` function, you can even see the number of palingrams found (2,230).

NOTE

The most common way to run `cProfile` is directly in the interpreter. This lets you dump your output to a text file and view it with a web viewer. For more information, visit **https://docs.python.org/3/library/profile.html**.

Profiling with time

Another way to time the program is to use `time.time()`, which returns an *epoch timestamp*—the number of seconds since 12 AM on January 1, 1970 UTC (the *Unix epoch*). Copy *palingrams.py* to a new file, save it as *palingrams_timed.py*, and insert the following code at the very top:

```
import time
start_time = time.time()
```

Now go to the end of the file and add the following code:

```
end_time = time.time()
print("Runtime for this program was {} seconds.".format(end_time - start_time))
```

Save and run the file. You should get the following feedback at the bottom of the interpreter window—give or take a few seconds:

```
Runtime for this program was 222.73954558372498 seconds.
```

The runtime is longer than before, as you are now evaluating the whole program, including printing, rather than just the `find_palingrams()` function.

Unlike `cProfile`, `time` doesn't provide detailed statistics, but like `cProfile`, it can be run on individual code components. Edit the file you just ran, moving the start and end time statements (as shown below in bold) so they bracket our long-running `find_palingrams()` function. Leave the `import` and `print` statements unchanged at the top and bottom of the file, respectively.

```
start_time = time.time()
palingrams = find_palingrams()
end_time = time.time()
```

Save and run the file. You should get the following feedback at the bottom of the interpreter window:

```
Runtime for this program was 199.42786622047424 seconds.
```

This now matches the initial results using `cProfile`. You won't get the exact same time if you rerun the program or use a different timer, but don't get hung up on it. It's the *relative* times that are important for guiding code optimization.

Palingram Optimization

I'm sorry, but three minutes of my life is too long to wait for palingrams. Armed with our profiling results, we know that the `find_palingrams()` function accounts for most of the processing time. This probably has something to do with reading and writing to lists, slicing over lists, or searching in lists. Using an alternative data structure to lists—like tuples, sets, or dictionaries—might speed up the function. Sets, in particular, are significantly faster than lists when using the `in` keyword. Sets use a hashtable for very fast lookups. With hashing, strings of text are converted to unique numbers that are much smaller than the referenced text and much more efficient to search. With a list, on the other hand, you have to do a linear search through each item.

Think of it this way: if you're searching your house for your lost cell phone, you could emulate a list by looking through every room before finding it (in the proverbial last place you look). But by emulating a set, you can basically dial your cell number from another phone, listen for the ringtone, and go straight to the proper room.

A downside to using sets is that the order of the items in the set isn't controllable and duplicate values aren't allowed. With lists, the order is preserved and duplicates are allowed, but lookups take longer. Fortunately for us, we don't care about order or duplicates, so sets are the way to go!

Listing 2-4 is the `find_palingrams()` function from the original *palingrams.py* program, edited to use a set of words rather than a list of words. You can find it in a new program named *palingrams_optimized.py*, which you can download from **https://www.nostarch.com/impracticalpython/**, or just make these changes to your copy of *palingrams_timed.py* if you want to check the new runtime yourself.

palingrams_optimized.py

```
def find_palingrams():
    """Find dictionary palingrams."""
    pali_list = []
 ❶ words = set(word_list)
 ❷ for word in words:
        end = len(word)
        rev_word = word[::-1]
        if end > 1:
            for i in range(end):
             ❸ if word[i:] == rev_word[:end-i] and rev_word[end-i:] in words:
                    pali_list.append((word, rev_word[end-i:]))
             ❹ if word[:i] == rev_word[end-i:] and rev_word[:end-i] in words:
                    pali_list.append((rev_word[:end-i], word))
    return pali_list
```

Listing 2-4: The `find_palingrams()` function optimized with sets

Only four lines change. Define a new variable, `words`, which is a set of `word_list` ❶. Then loop through the set ❷, looking for membership of word slices in this set ❸❹, rather than in a list as before.

Here's the new runtime for the `find_palingrams()` function in *palingrams_optimized.py*:

```
Runtime for this program was 0.4858267307281494 seconds.
```

Wow! From over three minutes to under a second! *That's* optimization! And the difference is in the data structure. Verifying the membership of a word in a *list* was the thing that was killing us.

Why did I first show you the "incorrect" way to do this? Because that's how things happen in the real world. You get the code to work, and then you optimize it. This is a simple example that an experienced programmer would have gotten right from the start, but it is emblematic of the overall concept of optimization: get it to work as best as you can, then make it better.

dnE ehT

You've written code to find palindromes and palingrams, profiled code using `cProfile`, and optimized code by using the appropriate data structure for the task. So how did we do with respect to Zatanna? Does she have a fighting chance?

Here I've listed some of the more "aggressive" palingrams found in the *2of4brif* dictionary file—everything from the unexpected *sameness enemas* to the harsh *torsos rot* to my personal favorite as a geologist: *eroded ore*.

dump mud	drowsy sword	sameness enemas
legs gel	denims mined	lepers repel
sleet eels	welsh slew	slam mammals
eroded ore	rise sir	pots nonstop
strafe farts	torsos rot	swan gnaws
wolfs flow	partner entrap	nuts stun
slaps pals	flack calf	knobs bonk

Further Reading

Think Python, 2nd Edition (O'Reilly, 2015) by Allen Downey has a short and lucid description of hashtables and why they are so efficient. It's also an excellent Python reference book.

Practice Project: Dictionary Cleanup

Data files available on the internet are not always "plug and play." You may find you need to massage the data a bit before applying it to your project. As mentioned earlier, some online dictionary files include each letter of the alphabet as a word. These will cause problems if you want to permit the use of one-letter words in palingrams like *acidic a*. You could always remove them by directly editing the dictionary text file, but this is tedious and for losers. Instead, write a short script that removes these af-

ter the dictionary has been loaded into Python. To test that it works, edit your dictionary file to include a few one-letter words like *b* and *c*. For a solution, see the appendix, or find a copy (*dictionary_cleanup_practice.py*) online at **https://www.nostarch.com/impracticalpython/**.

Challenge Project: Recursive Approach

With Python, there is usually more than one way to skin a cat. Take a look at the discussion and pseudocode at the Khan Academy website (**https://www.khanacademy.org/computing/computer-science/algorithms/recursive-algorithms/a/using-recursion-to-determine-whether-a-word-is-a-palindrome/**). Then rewrite the *palindrome.py* program so that it uses recursion to identify palindromes.